

IRMA

Conciencia Situacional Táctica — Documento de funcionamiento

Plataforma tipo **ATAK** reconstruida como aplicación web instalable (PWA) con transporte de malla **LoRa** de primera clase. Seguimiento de unidades en vivo, gráficos tácticos y chat sobre celular, Wi-Fi o radio fuera de red.

PWA offline-first

Next.js 15 · React 19

Leaflet

Zustand

LoRa / WebSocket / BLE / Serial

Generado el 2026-06-24 · Proyecto educativo / civil · No afiliado al TAK Product Center

1. Qué es y para qué sirve

IRMA da a un equipo de campo una imagen compartida en vivo: dónde está cada quién, qué marcan en el mapa y qué se dicen — funcione o no la red celular. Pensada para búsqueda y rescate (SAR), seguridad de eventos, expediciones y equipos de campo.

Capacidades clave

- **Mapa en vivo** (Leaflet, tiles oscuros CARTO) con flecha de posición propia + anillo de precisión GPS.
- **Seguimiento de equipo** por afiliación: aliado / hostil / neutral / desconocido, con rango, rumbo y batería.
- **Gráficos tácticos**: marcadores, líneas, polígonos, círculos, regla rango/rumbo — difundidos a toda la red.
- **Alertas de pánico** de un toque que parpadean en el mapa de todos.
- **Navegación a objetivo** (bloodhound) con brújula real del dispositivo.
- **Chat de red** + adjuntos (imagen / nota de voz) sobre el relay.

Transportes (un mismo app)

- **Malla simulada** — demo sin hardware.
- **Relay WebSocket** — amigos por internet, salas compartidas.
- **LoRa por USB serial** — Web Serial.
- **LoRa por Bluetooth LE** — Web Bluetooth (Nordic UART).

La abstracción **Transport** es la decisión de diseño clave: la app envía/recibe **Packet** sobre *cualquier* enlace sin cambiar UI ni estado.

2. Arquitectura general

Tres capas: **UI** (componentes React), **estado** (un store Zustand central) y **transporte** (enlace activo que mueve paquetes). El store es el único punto de verdad; la UI lo lee y el transporte lo alimenta.

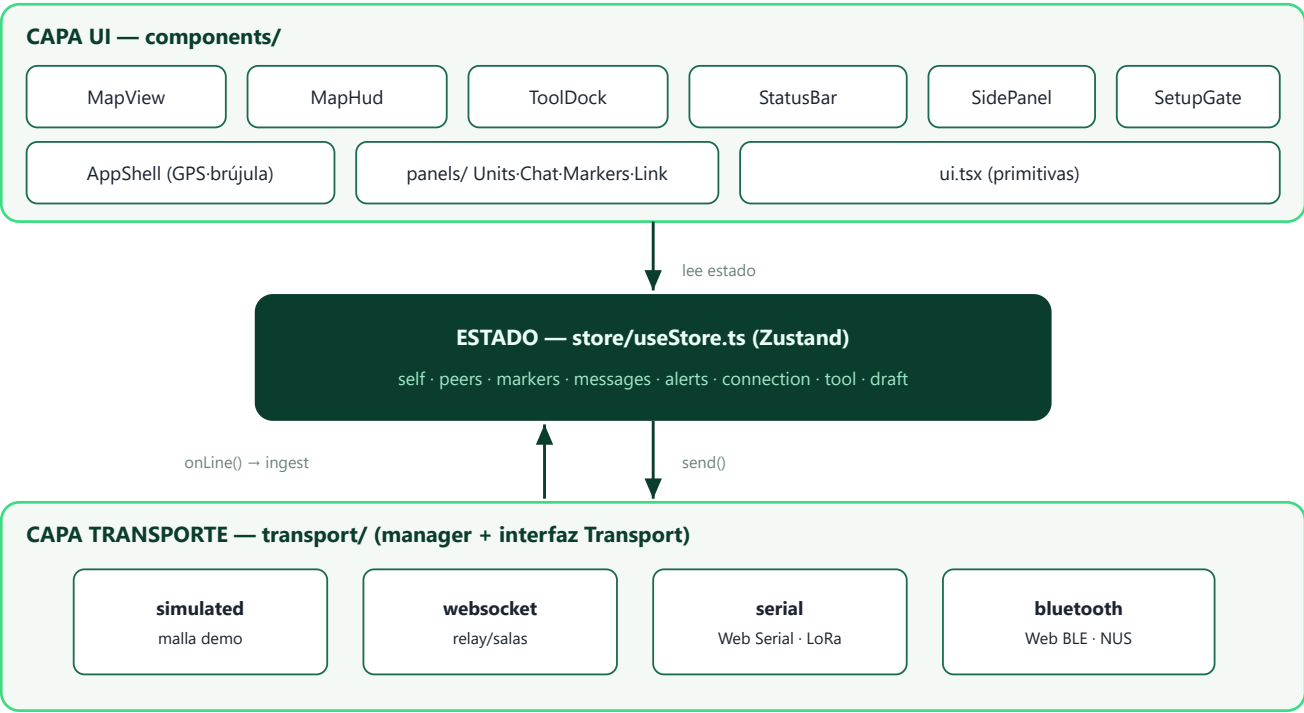


Figura 1 — Capas de la aplicación. La UI solo habla con el store; el store habla con el transporte activo.

Carpeta	Responsabilidad
lib/protocol/	packet.ts — códec compacto con guarda de MTU LoRa. cot.ts — interop Cursor-on-Target (ATAK/WinTAK).
lib/transport/	Interfaz Transport + 4 implementaciones + manager.ts (ciclo de vida del enlace activo).
lib/store/	useStore.ts — estado central Zustand: propio, pares, marcadores, mensajes, alertas, enlace.
lib/geo/	Haversine, rumbo, formateo, proyección.
components/	Mapa, HUD, dock de herramientas, barra de estado, panel lateral, gate de identidad.

3. Modelo de datos

Todo lo que viaja por la red y vive en el store se reduce a cuatro entidades.

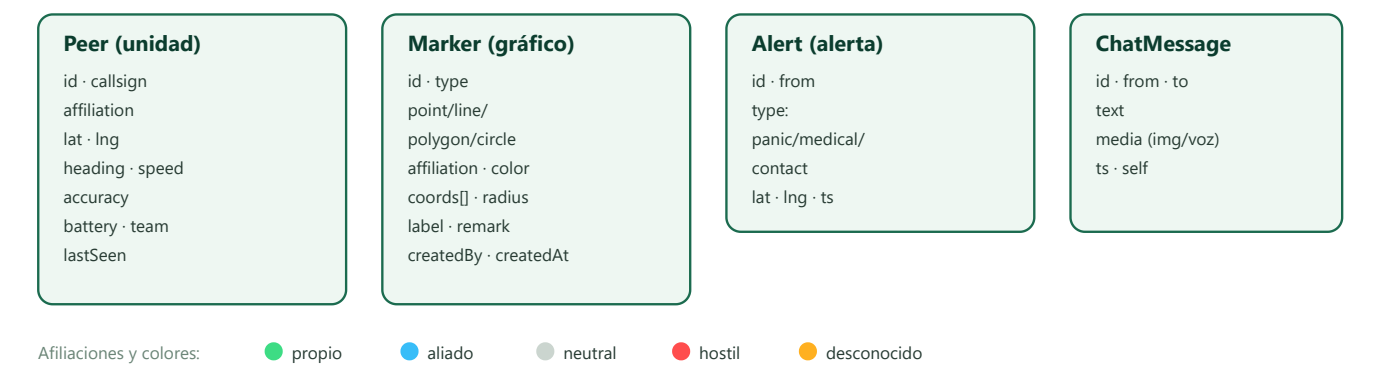


Figura 2 — Entidades del dominio (src/lib/types.ts).

4. Protocolo de cable

Un paquete = una línea de JSON compacto, dimensionada para caber en **una sola trama LoRa** (≤ 222 bytes , SF7–SF9). Claves cortas para abaratar el aire; `t` es el código de tipo.

```
// posición de RAVEN-1
{"t":0,"i":"a1b2","c":"RAVEN-1","a":"s","la":19.4326,"ln":-99.1332,"h":270,"b":88,"ts":1718480000000}
```

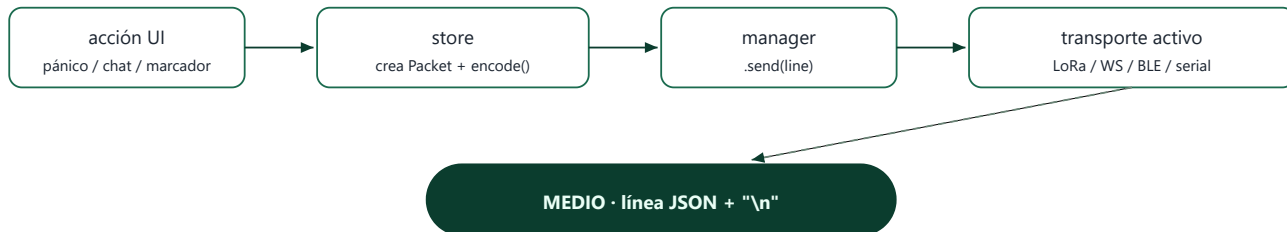
t	Tipo	Campos clave	LoRa?
0	posición	i id · c callsign · a afiliación · la / ln · h rumbo · s vel · ac precisión · b batería	sí
1	mensaje	i id · f de · o a (omitir = difusión) · x texto	sí
2	marcador	i · m tipo · a afiliación · co coords · rd radio · l etiqueta · r nota · by autor	sí
3	borrar-marcador	i id	sí
4	ping	i · f de · la / ln	sí
5	alerta	i · f de · k tipo (pánico/médico/contacto) · la / ln	sí
6	media	i · f de · mk img/audio · mm mime · d base64 · du duración	no (solo relay)

Códec en `src/lib/protocol/packet.ts`: `encode()` → línea, `decode()` → paquete, `fitsInFrame()` verifica el presupuesto de 222 B. Lat/lng se redondean a 6 decimales (5 para marcadores) para ahorrar bytes. Media (imagen/voz) excede de sobra una trama LoRa, así que **solo viaja por el relay WebSocket**.

5. Ciclo de vida del paquete (envío y recepción)

Toda mutación de la red pasa por el mismo embudo: el store serializa con `encode()` y empuja al `manager`; al recibir, el `manager` entrega líneas a `ingestLine()` que decodifica y aplica con `applyPacket()`.

SALIDA (TX)



ENTRADA (RX)



`applyPacket` ignora ecos del propio `id` y mensajes no dirigidos a uno; los pares viejos se purgan tras 5 min (`pruneStale`).

Figura 3 — Embudo único de salida y entrada de paquetes (`useStore.ts`).

Reglas de aplicación al recibir

- **position**: ignora el eco del propio `id`; un par que se anuncia como `self` se re-mapea a `friend` desde nuestra perspectiva.
- **message / media**: si lleva destinatario y no soy yo, se descarta; si no, entra al chat y suma no-leídos.
- **marker / marker-delete**: agrega o quita del diccionario de marcadores.
- **alert**: registra una baliza de socorro que parpadea en el mapa hasta limpiarla.
- **ping**: aparece como nota "© PING" en el chat.

6. Los cuatro transportes

El `TransportManager` posee un único enlace activo. Cambiar de enlace es `connect(kind)` : primero desmonta el anterior, luego instancia el nuevo. Todos cumplen la misma interfaz `Transport` (`connect` / `disconnect` / `send` + callbacks `onLine` / `onState` / `onLog`).

Transporte	Tecnología web	Alcance	Requisitos
Malla simulada	timers en la app	solo demo local	ninguno
Internet · relay	WebSocket	global, por salas	URL <code>wss://</code> + código de sala
LoRa · USB serie	Web Serial	fuera de red (radio)	Chromium + HTTPS/localhost + placa LoRa
LoRa · Bluetooth	Web Bluetooth (NUS)	fuera de red (radio)	Chromium + HTTPS/localhost + placa BLE

Ruta por internet — relay WebSocket

El relay (`relay/server.mjs`) es un **fan-out por sala agnóstico al formato**: reenvía cada línea que recibe a los *otros* clientes de la misma sala. No entiende el contenido — solo mueve líneas. Trama máxima 1 MB (cabe una imagen/voz comprimida).



Figura 4 — Topología del relay: difusión por sala. Despliega en Render/Railway/Fly y apunta `NEXT_PUBLIC_RELAY_URL=wss://tu-relay` .

Ruta fuera de red — puente LoRa

El navegador no maneja una radio LoRa directo. IRMA habla con un **puente**: cualquier placa LoRa (Heltec/TTGO/LilyGO, RAK, o un nodo Meshtastic) que retransmita **tramas delimitadas por salto de línea** entre su puerto USB/BLE y el PHY LoRa.



Figura 5 — Puente de radio. El firmware de referencia (Arduino, librería `LoRa` , SX127x) reenvía cada línea serial a la radio y emite cada trama recibida como línea.

7. Recorrido de uso (flujo del operador)

1. **Setup de identidad** (`SetupGate`): callsign, equipo, afiliación, código de sala y posición inicial. Gatea el mapa y persiste en `localStorage`.
2. **AppShell** arranca: vigila el GPS (`watchPosition`), lee la **brújula real** del dispositivo, late la posición propia (beacon periódico) y registra el service worker.
3. **Conectar un enlace** desde el panel **Enlace**: elige transporte → `connect()`. El estado pasa CONECTANDO → CONECTADO.
4. **Operar el mapa** con el `ToolDock`: colocar marcadores, dibujar líneas/polígonos/círculos, medir rango/rumbo, fijar posición a mano, centrar/seguir, emitir sirena.
5. **Difundir**: cada acción genera un `Packet` que sale por el enlace activo; los pares lo reciben y su mapa se actualiza.
6. **Navegar a objetivo**: selecciona unidad/marcador → HUD bloodhound con distancia + rumbo en vivo, flecha propia orientada por brújula.



Figura 6 — Flujo del operador de extremo a extremo.

Herramientas del mapa

Herramienta	Acción
Mano	Mover / seleccionar (clic en unidades y gráficos para detalle)
Marcadores ▾	Colocar marcador aliado / hostil / neutral / punto de ruta
Dibujar ▾	Trazar línea / polígono / círculo (se difunden a la red)
Regla	Rango + rumbo entre dos clics (Haversine)
Fijar posición	Toca el mapa para fijar tu ubicación a mano (sobrevive recargas)
Centrar	Recentrar y seguir tu posición
Sirena	Emitir alerta de pánico a la sala (parpadea en todos los mapas)

8. PWA, persistencia y offline

Instalable + offline-first

Manifest + iconos la hacen instalable. El service worker (`public/sw.js`, solo en producción) cachea el app-shell con estrategia **network-first** y los tiles/glyphs con **cache-first**: un área ya vista queda disponible fuera de red.

Persistencia local

La **identidad** (callsign, equipo, sala, afiliación, posición fijada a mano) y la **sesión** (marcadores + chat) se guardan en `localStorage`. Una suscripción al store auto-graba cuando cambian marcadores o mensajes. Las posiciones GPS en vivo son efímeras; solo la posición fijada a mano se restaura.

9. Stack y cómo correrlo

[Next.js 15 \(App Router\)](#)[React 19](#)[TypeScript](#)[Tailwind CSS v4](#)[Leaflet](#)[Zustand](#)[ws \(relay\)](#)

```
# desarrollo local
npm install
npm run dev          # http://localhost:3000

# celular con GPS real (HTTPS en la LAN)
npm run dev:lan      # abre https://<ip-LAN>:3000 en el teléfono

# producción (service worker / offline activo)
npm run build && npm start

# relay multi-dispositivo por internet
npm run relay        # escucha en :1234 (o $PORT)
```

Despliegue: app en Vercel, relay en cualquier host con proceso de larga vida + WebSocket (Render/Railway/Fly; incluye `render.yaml` y `relay/Dockerfile`). Apunta la app con `NEXT_PUBLIC_RELAY_URL=wss://tu-relay`.

10. Hoja de ruta

- Transporte Meshtastic protobuf nativo (directo, sin firmware custom).
- Cifrado E2E de paquetes (claves por equipo).
- Cuadrícula MGRS y entrada de coordenadas.
- Paquetes de tiles offline (importación MBTiles).
- Rastros (breadcrumbs) e historial de unidades.
- Geofencing y alertas de zona. · Llamadas de voz (WebRTC).

IRMA — proyecto educativo / civil de conciencia situacional. No afiliado al TAK Product Center ni al Gobierno de EE. UU. · Documento generado a partir del código fuente (README.md, src/lib/, relay/server.mjs).